

Inter-Service Communication: REST & Message Queues

1. What is Inter-Service Communication?

In a microservices architecture, services need to communicate with each other to complete a business workflow.

Example:

```
Order Service
  ↓
Payment Service
  ↓
Payment Gateway
```

The communication between these services is called **inter-service communication**.

There are two major approaches:

```
Inter-Service Communication
|
├── Synchronous
│   └── REST / HTTP
|
└── Asynchronous
    ├── Message Queue / Broker
    │   ├── Kafka
    │   ├── RabbitMQ
    │   └── ActiveMQ
```

2. REST Communication

REST-based communication uses HTTP request/response.

Example:

```
Service A                Service B
|                         |
|----- HTTP Request ----->|
|                         |
|<----- HTTP Response -----|
```

Service A normally **waits for the response** before continuing.

Example

```
POST /payments
```

Request:

```
{
  "customerId": "C1001",
  "amount": 5000
}
```

Response:

```
{
  "transactionId": "TX123",
  "status": "SUCCESS"
}
```

3. REST Clients in Spring Boot

You will commonly encounter:

RestTemplate

Synchronous/blocking HTTP client.

```
ResponseEntity<PaymentResponse> response =
    restTemplate.exchange(
        url,
        HttpMethod.POST,
        entity,
        PaymentResponse.class
    );
```

OpenFeign

Declarative HTTP client.

```
@FeignClient(name = "payment-service")
public interface PaymentClient {

    @PostMapping("/payments")
    PaymentResponse createPayment(
        @RequestBody PaymentRequest request);
}
```

Usage:

```
paymentClient.createPayment(request);
```

WebClient

Spring's HTTP client that supports reactive/non-blocking communication.

```
webClient.post()
    .uri("/payments")
    .bodyValue(request)
    .retrieve()
    .bodyToMono(PaymentResponse.class);
```

For your project, **BPS actually uses RestTemplate for Hydra/Chase**, so that should be your primary example.

4. Why Use REST?

REST is a good choice when the caller **needs the response immediately**.

For example:

```
BPS
↓
Hydra
↓
SUCCESS
↓
BPS continues processing
```

BPS can't simply move on without knowing the payment-provider result.

This is why synchronous REST is a natural fit for many online payment workflows.

5. Advantages of REST

Simple

Request → response is easy to understand.

Immediate result

The caller gets the downstream result directly.

Easy debugging

You can inspect:

```
URL
Headers
Request
Response
HTTP status
```

Technology independent

Java can call .NET, Python, Go, etc., as long as they expose HTTP APIs.

6. Problems With REST

The major problem is **runtime dependency**.

Suppose:

```
BPS → Hydra
```

and Hydra is down:

BPS → Hydra ✕

Now BPS has to handle:

- connection timeout
- read timeout
- 4xx/5xx
- retry
- circuit breaker
- failure response
- possibly unknown transaction state

REST therefore introduces **synchronous coupling**.

7. Payment Example: Timeout

This is particularly important for your BPS interview.

Suppose:

```
BPS
↓
Hydra
↓
Payment successfully processed
↓
Response lost
```

BPS sees:

TIMEOUT

It would be wrong to automatically conclude:

Payment FAILED

The payment may already have succeeded.

So you need to think about:

```
Timeout
↓
Transaction state
↓
Idempotency
↓
Retry decision
↓
Reconciliation
```

This is an excellent payment-system interview scenario.

8. Message Queue / Message Broker

A message queue changes the communication model.

Instead of:

```
Service A → Service B
```

you have:

```
Service A
  ↓
Message Broker
  ↓
Service B
```

Service A publishes a message and does not necessarily wait for Service B to finish processing it.

Example:

```
Payment Service
  ↓
PaymentCompleted event
  ↓
Kafka
  ↓
Notification Service
```

9. Producer and Consumer

Producer

The service that sends/publishes the message.

```
BPS → Producer
```

Consumer

The service that reads/processes the message.

```
Consumer ← Kafka
```

Example:

```
BPS
  ↓
Producer
  ↓
Kafka
  ↓
History Consumer
```

10. Why Use Messaging?

Messaging is useful when the work can happen **asynchronously**.

For example, after a successful payment:

```
Payment completed
  ↓
Need to:
├─ send notification
├─ update analytics
└─ create audit event
```

These don't necessarily need to block the payment response.

You could publish:

```
PaymentCompleted
```

and allow multiple consumers to process it independently.

11. REST vs Message Queue

REST	Message Queue
Synchronous request/response	Asynchronous message processing
Caller waits	Caller can continue
Direct service-to-service communication	Broker between services
Immediate response	Processing can happen later
Higher runtime dependency	Lower runtime dependency
Simpler	More complex
Good for immediate result	Good for decoupled/background work

The important distinction is:

REST is generally “call and wait”; messaging is generally “send and process later.”

12. Message Queue Does Not Mean Faster

Don't make this interview mistake:

“Kafka is faster than REST.”

Not necessarily.

Messaging provides benefits like:

- decoupling
- buffering
- asynchronous processing
- independent consumer scaling

But the actual business processing may happen later.

13. Queue vs Topic

Queue

Typically, a message is processed by one competing consumer.

```
Queue
  ↓
Consumer A
Consumer B
Consumer C
```

For a given message, one consumer generally handles it.

Topic

Multiple independent consumer groups can consume the same event.

```
PaymentCompleted
  ↓
  Topic
 /  |  \
↓   ↓   ↓
Audit History Notification
```

Kafka primarily uses **topics, partitions and consumer groups**.

14. Important Messaging Concepts

At your experience level, know these:

Partition

Kafka divides a topic into partitions for scalability and parallelism.

```
Topic
├─ Partition 0
├─ Partition 1
└─ Partition 2
```

Consumer Group

Consumers working together to process a topic.

```
Payment Topic
  ↓
Consumer Group
├─ Consumer 1
├─ Consumer 2
└─ Consumer 3
```

Each partition is assigned to at most one consumer within a consumer group at a given time.

Offset

A position representing progress through a partition.

15. At-Least-Once Delivery

A common messaging model is **at-least-once delivery**.

That means a message can potentially be delivered more than once.

```
PaymentCompleted
  ↓
Consumer
  ↓
processing fails/ack isn't committed
  ↓
message delivered again
```

Therefore consumers should often be **idempotent**.

16. Idempotent Consumer

Suppose:

```
Event ID = EVT100
```

arrives twice.

Your consumer should make sure:

```
EVT100 → processed once
```

rather than:

```
EVT100 → history inserted
EVT100 → history inserted again
```

A typical approach is to track a unique event/transaction ID.

This is particularly important for payment-related processing.

17. Retry and DLQ

Suppose:

```
Kafka
  ↓
Consumer
  ↓
Database unavailable
```

You may retry:

```
Attempt 1 → fail
Attempt 2 → fail
Attempt 3 → fail
```

After the configured retry policy, the message may go to a:

Dead Letter Queue / Dead Letter Topic

```
Main Topic
  ↓
Consumer
  ↓
Retries
  ↓
DLQ
```

Operations teams can investigate or reprocess it.

18. Ordering

Suppose you receive:

```
PAYMENT_INITIATED
PAYMENT_SUCCESS
PAYMENT_REVERSED
```

The sequence may matter.

If the consumer sees:

```
PAYMENT_SUCCESS
PAYMENT_INITIATED
PAYMENT_REVERSED
```

that could create an invalid business state.

Kafka guarantees ordering **within a partition**, not globally across all partitions.

This is an important interview detail.

19. REST Failure Handling

For REST:

```
BPS
  ↓
Hydra
```

you need to consider:

Timeout

```
Hydra doesn't respond
```

Retry

Should we try again?

Circuit breaker

Hydra repeatedly failing

↓

Stop sending calls temporarily

Idempotency

Don't accidentally process payment twice

Reconciliation

Determine actual payment state later

20. Messaging Failure Handling

Messaging has a different failure model:

```
Producer
↓
Broker ✓
↓
Consumer ✕
```

The message can remain available for retry depending on the broker and consumer semantics.

So the broker provides **durable buffering**, subject to its configuration and guarantees.

But now you have new concerns:

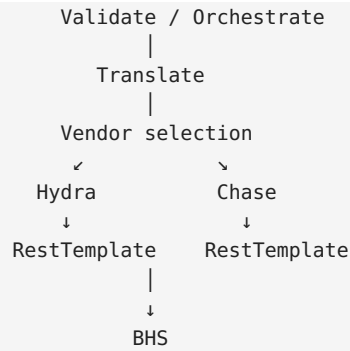
```
Duplicate messages
Ordering
Consumer retries
DLQ
Offset management
Poison messages
Idempotency
```

21. Where Does BPS Fit?

Now let's apply the concept to **your actual BPS project**.

Your architecture is:

```
BASAPI ─┐
AF ───┐  ──> BPS
Majesco ─┘
      |
```



BPS is the **bridge between the billing system and payment infrastructure**.

It:

```
Validate
↓
Orchestrate
↓
Translate
↓
Call payment provider
↓
Post to Majesco
↓
Reconcile
↓
Store history through BHS
```

22. BPS → Hydra/Chase

This is your most important real REST example.

The token contains:

```
vendor
```

BPS uses that information with **Factory + Strategy**.

Conceptually:

```
Token
↓
vendor = HYDRA
↓
Factory
↓
Hydra Strategy
↓
RestTemplate
↓
Hydra
```

Or:

```
Token
↓
vendor = CHASE
```

```
↓
Factory
↓
Chase Strategy
↓
RestTemplate
↓
Chase
```

So you can explain that BPS uses **REST synchronously to communicate with payment infrastructure**.

23. Why Is RestTemplate Appropriate in BPS?

Based on your implementation:

```
BPS
↓
RestTemplate
↓
Hydra/Chase
↓
response
↓
BPS continues orchestration
```

BPS needs the external payment response to continue the workflow.

So the synchronous model fits the online payment process.

24. BPS Isn't Just an HTTP Proxy

This is an important point to communicate in your interview.

BPS is not simply:

```
Source → BPS → Hydra
```

It is doing substantial business processing:

```
Source
↓
BPS
├─ Validate
├─ Identify vendor
├─ Select strategy
├─ Translate request
├─ Invoke payment provider
├─ Translate response
├─ Post to Majesco
├─ Reconcile
└─ History through BHS
```

Therefore describe BPS as an **orchestration and translation layer / bridge**, not merely an API

forwarding layer.

25. What About Message Queues in BPS?

You told me BPS's recurring payment processing uses **sequential chunk-based batch processing**, and you haven't indicated that this workflow uses Kafka/RabbitMQ.

So don't say:

“BPS uses Kafka for recurring payments.”

Instead:

```
Online payments
→ synchronous REST

Recurring payments
→ batch processing
→ sequential chunks
```

A message broker could theoretically be introduced for asynchronous processing, but that would be a **different architecture**, not your current BPS implementation.

26. Could BPS Use Messaging?

Yes, theoretically.

For example, suppose after successful payment you need history processing that does not have to block the response:

```
BPS
↓
Payment SUCCESS
↓
PaymentCompleted event
↓
Message Broker
↓
BHS consumer
```

But this changes the consistency model.

Instead of:

```
BPS → BHS
```

you may have:

```
BPS → Broker → BHS
```

Now you need to think about:

- duplicate events
- eventual consistency

- retries
- DLQ
- event ordering
- correlation IDs

So messaging isn't automatically an improvement.

27. BPS Scenario: Hydra Timeout

This is a scenario you should practice explaining.

```

BASAPI
↓
BPS
↓
Factory
↓
Hydra Strategy
↓
RestTemplate
↓
Hydra
↓
TIMEOUT

```

What should BPS do?

Don't immediately say:

▮ "Retry the payment."

First determine:

```

Did Hydra receive/process it?
↓
Unknown?
↓
Check status / reconciliation

```

Then decide whether retry is safe.

This is where your **REST + exception handling + idempotency + reconciliation** knowledge connects.

28. BPS Scenario: BHS Failure

Suppose:

```

BPS
├── Hydra ✓
└── BHS ✗

```

The correct behavior depends on your actual business requirement.

Questions to determine:

Is BHS history mandatory for payment success?

Can BHS operation be retried?

Is payment state already durable elsewhere?

Can reconciliation repair the missing history?

This is a good interview answer because you're not pretending there is one universal solution.

29. BPS Recurring Payment Scenario

Your recurring flow:

```
Recurring payment records
    ↓
Batch
    ↓
Sequential Chunk 1
    ↓
Process
    ↓
Sequential Chunk 2
    ↓
Process
    ↓
...
```

This demonstrates another concept:

Batch processing and message-based asynchronous processing are different design approaches.

Chunking controls how much work is processed at a time.

Messaging decouples producers and consumers.

They can be combined in some architectures, but they aren't synonyms.

30. Interview Questions You Should Prepare

General

1. What is inter-service communication?
2. REST vs message queue?
3. Synchronous vs asynchronous?
4. What is `RestTemplate`?
5. What is Feign?
6. What is WebClient?
7. Why would you choose REST?
8. Why would you choose messaging?

REST resilience

9. What happens when the downstream service times out?

10. How do you configure timeout?
11. When should you retry?
12. Why is retry dangerous for payment APIs?
13. What is circuit breaker?
14. What is idempotency?

Messaging

15. What is Kafka?
16. Producer vs consumer?
17. Queue vs topic?
18. What is partition?
19. What is consumer group?
20. What is offset?
21. What is at-least-once delivery?
22. Why do consumers need idempotency?
23. What is DLQ?
24. How do you handle message ordering?

BPS-specific

25. How does BPS communicate with Hydra?
26. How does BPS communicate with Chase?
27. Why does BPS use `RestTemplate`?
28. How does BPS select the vendor?
29. Why use Strategy + Factory?
30. How does BPS translate the request?
31. What happens when Hydra times out?
32. What happens when Hydra succeeds but the response is lost?
33. What happens if BHS fails?
34. How does reconciliation help?
35. How is recurring-payment processing different from online payment?
36. Why use sequential chunks for recurring payments?

31. Strong Interview Answer

If asked:

“Explain inter-service communication and how it works in your project.”

You can say:

“Inter-service communication allows different backend services to exchange data and participate in a business workflow. The two common models are synchronous HTTP-based communication, such as REST, and asynchronous communication through a message broker such as Kafka or RabbitMQ. REST is appropriate when the caller needs the downstream response immediately, while messaging is useful when processing can be decoupled and performed asynchronously.

In BPS, our payment-provider integration is synchronous REST-based. BPS uses `RestTemplate` to communicate with Hydra and Chase. The token contains a vendor field, and we use Factory and Strategy patterns to select the appropriate vendor implementation, which then makes the REST call. BPS validates, orchestrates and translates the request and response, performs the required posting/reconciliation with Majesco, and interacts with BHS

for history. For recurring payments, which are separate from online payments, BPS uses sequential chunk-based batch processing rather than a message-queue model.”

The mental model to remember

REST

"Call another service and wait for its response."

Message Queue

"Send a message and let another service process it asynchronously."

RestTemplate

"How BPS currently makes its HTTP calls."

Feign

"Declarative alternative for HTTP service clients."

WebClient

"Reactive/non-blocking HTTP client."

BPS

"Validate → Orchestrate → Translate → Vendor → Payment → Post/Reconcile → History."

For **your 3-year interview**, the most important chain to master is:

REST → RestTemplate → timeout → retry → idempotency → circuit breaker → reconciliation → REST vs messaging trade-offs.